

Translating from ORMS to SQLAlchemy Lab

- Due No Due Date
- Points 1
- Submitting a website url

FORK

[_ \(https://github.com/learn-co-curriculum/python-p3-translating-from-orms-to-sqlalchemy-lab/fork\)](https://github.com/learn-co-curriculum/python-p3-translating-from-orms-to-sqlalchemy-lab/fork)[curriculum/python-p3-translating-from-orms-to-sqlalchemy-lab](https://github.com/learn-co-curriculum/python-p3-translating-from-orms-to-sqlalchemy-lab)[_ \(https://github.com/learn-co-curriculum/python-p3-translating-from-orms-to-sqlalchemy-lab/issues/new\)](https://github.com/learn-co-curriculum/python-p3-translating-from-orms-to-sqlalchemy-lab/issues/new)

Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
- Use SQLAlchemy to create, read, update and delete records in a SQL database.

Key Vocab

- **Schema:** the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **Persist:** save a schema in a database.
- **Engine:** a Python object that translates SQL to Python and vice-versa.
- **Session:** a Python object that uses an engine to allow us to programmatically interact with a database.
- **Transaction:** a strategy for executing database statements such that the group succeeds or fails as a unit.
- **Migration:** the process of moving data from one or more databases to one or more target databases.

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `lib/` folder.

The testing file for this lab will execute many of the same tests as *"Putting it All Together: ORMs Lab"* from the previous Canvas module. There are nine tests in total: one for your data model in `lib.models.py` and eight for your functions in `lib/dog.py`. In the previous lab, you had to write quite a bit of code to get those tests passing- using SQLAlchemy, it should be much easier.

Tips and Tricks

- The bodies of all functions in `dog.py` except `create_table()` and `save()` should be composed of a single line of code.
- Read through the `pytest` error messages to make sure the input and output for your functions match the tests.
- Remember which attributes are required when designing a SQLAlchemy data model: a `__tablename__`, a `primary_key`, and one or more `Column`s.

Once all of your tests are passing, commit and push your work using `git` to submit.

Resources

- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/)  `(https://docs.sqlalchemy.org/en/14/orm/)`
- [SQLAlchemy ORM Session Basics](https://docs.sqlalchemy.org/en/14/orm/session_basics.html)  `(https://docs.sqlalchemy.org/en/14/orm/session_basics.html)`
- [SQLAlchemy ORM Column Elements and Expressions](https://docs.sqlalchemy.org/en/14/core/sqlelement.html)  `(https://docs.sqlalchemy.org/en/14/core/sqlelement.html)`
- [SQLAlchemy ORM Querying Guide](https://docs.sqlalchemy.org/en/14/orm/queryguide.html)  `(https://docs.sqlalchemy.org/en/14/orm/queryguide.html)`